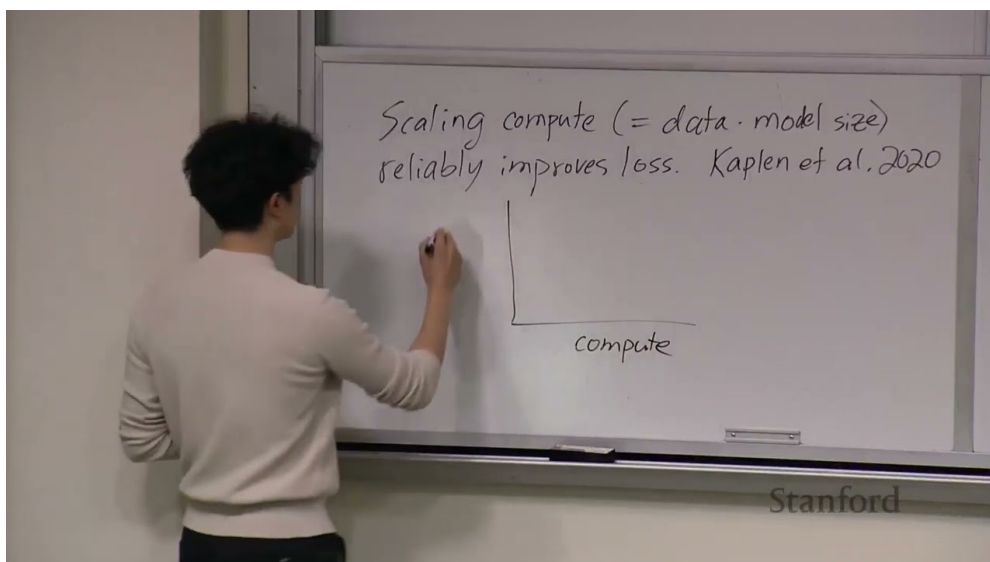


课程笔记

[CS25] Intuitions on LMs + History of Architectures — Jason Wei + Hyung Won Chung, OpenAI

五道口纳什 & Codex

2026 年 3 月 29 日



视频作者/频道: Stanford CS25: Transformers United

发布日期: Spring 2024

视频时长: ~1h

视频链接: 未填写

目录

1 引言	2
2 Part I: 语言模型的直觉 (Jason Wei)	2
2.1 手动检查数据的重要性	2
2.2 互联网数据中蕴含的能力	2
2.3 指令微调的直觉	2
2.4 思维链的本质	2
2.5 本章小结	3
3 Part II: 架构演化史 (Hyung Won Chung)	3
3.1 苦涩的教训 (The Bitter Lesson)	3
3.2 归纳偏置的角色	3
3.3 从 Encoder-Decoder 到 Decoder-Only	3
3.3.1 Encoder-Decoder 时代	3
3.3.2 BERT vs GPT 的分歧	3
3.3.3 Decoder-Only 的工程优势	4
3.4 训练目标的演化	4
3.5 本章小结	4
4 总结与延伸	4
4.1 拓展阅读	4

1 引言

本讲由两位 OpenAI 研究科学家联合主讲。Jason Wei（前 Google Brain）以“为什么语言模型如此有效”为主题，分享了从数据视角理解 LLM 的直觉；Hyung Won Chung 则从架构演化历史出发，讨论归纳偏置与规模化的深层关系。

2 Part I: 语言模型的直觉 (Jason Wei)

2.1 手动检查数据的重要性

理解模型的第一步是理解数据

Jason Wei 强调：理解 LLM 为何有效的最佳工具是“手动检查训练数据”。他分享了一个亲身经历——在训练肺癌分类器之前，他先花时间学会了识别不同类型的肺癌。这种“先理解数据，再训练模型”的方法论贯穿了他的整个职业生涯。

2.2 互联网数据中蕴含的能力

Jason 分析了预训练数据（互联网文本）中天然包含的各类“任务”：

- **翻译**：多语言网页、多语言 Wikipedia
- **代码生成**：GitHub、Stack Overflow
- **推理**：数学论坛、科学论文
- **对话**：Reddit 评论、论坛讨论
- **事实知识**：百科全书、新闻

语言建模

下一个 token 预测看似简单，但为了准确预测下一个 token，模型需要隐式地学习语法、语义、逻辑推理、世界知识等多种能力。训练数据的多样性直接决定了模型能力的广度。

2.3 指令微调的直觉

指令微调激活而非创造能力

指令微调（Instruction Tuning）的作用不是教模型新知识，而是教模型如何“格式化”已有知识来响应用户指令。FLAN 系列工作表明：在大量任务描述上微调可以显著提升零样本泛化能力。

2.4 思维链的本质

思维链提示之所以有效，是因为预训练数据中包含大量“展示推理过程”的文本（如数学解题步骤、代码注释）。CoT 本质上是在推理时激活了模型在预训练中学到的“分步思考”模式。

2.5 本章小结

LLM 的能力根植于训练数据的多样性；指令微调和提示工程是释放这些能力的钥匙。

3 Part II：架构演化史 (Hyung Won Chung)

3.1 苦涩的教训 (The Bitter Lesson)

Rich Sutton 的苦涩教训

AI 研究 70 年的核心经验：依赖人类知识的方法最终都会被纯粹的计算规模化方法超越。搜索和学习是两种能够有效利用更多计算资源的通用方法，而领域特定的工程技巧虽然短期有效，但长期阻碍扩展。

3.2 归纳偏置的角色

Hyung Won 提出了一个统一框架来理解架构演化：

归纳偏置是计算不足时的权宜之计

在给定的计算资源、数据和算法条件下，存在一个最优的“结构添加量”（归纳偏置）。计算充足时，应尽量减少归纳偏置，让模型自主学习结构。但学术界擅长“添加结构”（发论文），却不擅长“移除结构”（没有激励）。

3.3 从 Encoder-Decoder 到 Decoder-Only

3.3.1 Encoder-Decoder 时代

原始 Transformer (2017) 使用 encoder-decoder 架构。编码器使用双向注意力，解码器使用因果注意力，两者通过交叉注意力连接。

- **编码器**：双向注意力，能看到全部输入上下文
- **解码器**：因果注意力，只能看到已生成的 token
- **交叉注意力**：解码器 token 关注编码器输出

3.3.2 BERT vs GPT 的分歧

- **BERT** (2018)：仅使用编码器 + 双向注意力 + 掩码语言模型目标
- **GPT** (2018)：仅使用解码器 + 因果注意力 + 自回归目标

双向注意力的陷阱

BERT 的双向注意力在 2018 年效果更好，但这是一种“结构性捷径”——它假设我们可以看到整个输入。然而随着规模扩大和生成式任务的兴起，decoder-only 架构通过纯规模化超越了编码器方法。双向注意力是一种归纳偏置，在规模化时反而成为障碍。

3.3.3 Decoder-Only 的工程优势

从推理效率角度看，decoder-only 架构支持 KV 缓存——在多轮对话中，已处理的 token 的注意力可以被缓存和复用，而双向注意力不支持此优化。

3.4 训练目标的演化

- 掩码语言模型 (BERT): 预测被掩码的 token $\rightarrow$ 需要双向上下文
- 前缀语言模型 (T5): 前缀双向 + 后续自回归
- 自回归语言模型 (GPT): 纯因果预测 $\rightarrow$ 最少假设, 最好扩展

3.5 本章小结

架构从复杂走向简单的演化路径，恰好印证了“苦涩的教训”：减少归纳偏置 + 增加计算 = 更好的结果。

4 总结与延伸

两位讲者从不同角度达成了统一结论：LLM 的成功源于“简单方法 + 规模化”。Jason 从数据角度说明了预训练数据的隐式多任务性质；Hyung Won 从架构角度说明了减少人为假设的重要性。对未来的展望包括：定制化硬件（将确定的架构硬编码到芯片）和继续移除不必要的归纳偏置。

4.1 拓展阅读

- Sutton, “The Bitter Lesson”, 2019
- Wei et al., “Finetuned Language Models Are Zero-Shot Learners” (FLAN), 2022
- Chung et al., “Scaling Instruction-Finetuned Language Models” (Flan-T5/PaLM), 2022
- Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers”, 2019